ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP THỰC HÀNH
ỨNG DỤNG AI VÀ GIT PHÂN TÍCH REPOSITORY ĐỒ ÁN

Phân tích, đối sánh và phản biện sức khỏe kỹ thuật của các Repository Bài tập lớn môn Nhập môn Công nghệ Phần mềm

Học phần: IT3180 - Nhập môn Công nghệ phần mềm

Học kỳ: 2026.1

Chủ đề thực hành: Đọc cấu trúc Repo, Quy trình Git & Đánh giá đóng góp thành viên

Đối tượng phân tích:

- **Repo 1:** `github.com/neveraccept/IT3180_2025.2_Nhom7` (Nhóm 7)
- **Repo 2:** `github.com/khanh201104/Bai_tap_lon_IT3180_nhom_4` (Nhóm 4)

Phương pháp luận: Theo chuẩn tài liệu hướng dẫn *IT3180_0505_C5.5*

Hà Nội, Học kỳ 2026.1

TÓM TẮT BÁO CÁO (EXECUTIVE SUMMARY)

Báo cáo này trình bày quy trình kỹ thuật kết hợp giữa **công cụ quản lý mã nguồn Git** và **trợ lý Trí tuệ nhân tạo (AI)** để thẩm định chất lượng kỹ thuật, quy trình làm việc và mức độ đóng góp thành viên của hai repository BTL kỳ trước môn IT3180.

Thông qua việc đóng vai trò là sinh viên nhóm trưởng thực hiện thiết kế các prompt chuyên biệt (Role Prompting, Task Extraction Prompting, và Self-Critique Prompting) và nạp dữ liệu Git log, cây thư mục thực nghiệm trích xuất từ 2 repository, báo cáo làm sáng tỏ:

- **Khía cạnh kiến trúc & tổ chức:** Sự khác biệt rõ rệt giữa mô hình phân tầng decoupled hiện đại (Frontend React + Backend REST API của Nhóm 7) và mô hình Monolith MVC truyền thống (Spring Boot + Thymeleaf của Nhóm 4).
- **Khía cạnh tài liệu hóa:** Cả hai nhóm đều chuẩn bị tài liệu rất tốt; Nhóm 7 vượt trội với tài liệu đặc tả kiến trúc phần mềm `docs/SDD.md` dài 191 KB (12 chương), trong khi Nhóm 4 tổng hợp toàn bộ SRS vào 1 file `README.md` dài 1146 dòng.
- **Khía cạnh kiểm thử & CI/CD:** Điểm yếu chí mạng phổ biến ở cả 2 nhóm khi tỷ lệ bao phủ kiểm thử (Test Coverage) là **0%** (chỉ có file test rỗng sinh tự động `contextLoads()`) và hoàn toàn vắng bóng quy trình tích hợp liên tục CI/CD.
- **Khía cạnh quy trình Git & Đóng góp:** Phân tích phản biện 4 cạm bẫy thực tế: biến động dòng code (LOC) ảo do template giao diện, lẫn lộn danh tính qua email/tên, file `.gitignore` sơ sài, và hiện tượng "One-man army" / commit hộ.

PHẦN 1: THIẾT KẾ BỘ PROMPT AI CHUẨN HÓA

Để đảm bảo AI không đưa ra các nhận định chung chung, hời hợt hoặc bịa đặt (hallucination), quy trình được thiết kế thành hệ thống prompt có cấu trúc chặt chẽ theo hướng dẫn của Bộ môn.

1.1. Prompt 0 — Thiết lập vai trò & Ràng buộc cốt lõi (Bắt buộc chạy trước)

Thiết lập ngữ cảnh chuyên gia CNPM và cài đặt 4 ràng buộc đạo đức/kỹ thuật nhằm loại bỏ thiên kiến:

```
Bạn là chuyên gia Công nghệ phần mềm, hỗ trợ giảng viên/nhóm sinh viên phân tích repo đồ án môn
IT3180. Quy tắc bắt buộc trong suốt phiên làm việc: 1. Mọi nhận xét PHẢI kèm bằng chứng cụ thể
(đường dẫn file, mã commit SHA, số liệu con số, tên nhánh). Không có bằng chứng thì ghi rõ "không
đủ dữ liệu". 2. TUYỆT ĐỐI không tự xếp hạng thành viên "giỏi hơn / lười hơn". Chỉ trình bày dữ
liệu và nêu câu hỏi cần hỏi nhóm để làm rõ. 3. Phân biệt rõ "dữ liệu Git nói gì" với "suy đoán
của bạn". 4. Trả lời bằng tiếng Việt, súc tích, dùng bảng khi phù hợp.
```

1.2. Prompt 1 — Phân tích Cấu trúc thư mục & Tổ chức Repo

```
Đính kèm: (1) Cấu trúc thư mục (tree -L 3), (2) Danh sách file Git quản lý (git ls-files), (3)
Nội dung README.md. Hãy phân tích và trả về: 1. KIẾN TRÚC: Tóm tắt dự án theo tầng/thành phần
(frontend, backend, CSDL, các lớp code). Mỗi tầng chỉ rõ nằm ở thư mục nào. 2. ƯU ĐIỂM về cách tổ
chức — mỗi điểm kèm thư mục/file làm bằng chứng. 3. THÀNH PHẦN TỐT NÊN CÓ MÀ ĐANG THIẾU (Kiểm tra
cụ thể: Thư mục test thật/rỗng? CI/CD? .gitignore có đủ không, có file build/node_modules commit
nhầm không? Tài liệu kiến trúc & hướng dẫn chạy?). 4. ĐIỂM "ĐỘ TRƯỞNG THÀNH KỸ THUẬT" /10 kèm
bảng chấm và lý giải ngắn.
```

1.3. Prompt 2 — Xử lý trùng lặp danh tính tác giả (.mailmap)

```
Đính kèm danh sách tác giả (git shortlog -sne --all --no-merges) và bảng ánh xạ email-tên (git
log --format='%ae | %an'). Hãy: 1. Phát hiện các danh tính có thể là CÙNG MỘT NGƯỜI (trùng tên,
email tương tự, địa chỉ noreply GitHub). Lập bảng "danh tính gốc -> các bí danh". 2. Sinh nội
dung file .mailmap để gộp các bí danh đó. 3. Cảnh báo các trường hợp mơ hồ cần tự xác nhận với
nhóm.
```

1.4. Prompt 3 — Phân tích Đóng góp thành viên & Nhịp làm việc

```
Đính kèm git log numstat đầy đủ (git log --all --no-merges --numstat --date=short --
pretty=format:'COMMIT|%h|%an|%ae|%ad|%s') cùng bảng .mailmap đã gộp. Hãy trả về: 1. BẢNG TỔNG HỢP
mỗi thành viên: số commit, tổng dòng thêm, tổng dòng xóa, dòng net, số file từng chạm, khoảng
thời gian hoạt động. 2. PHÂN LOẠI theo LOẠI công việc dựa trên đường dẫn file: Backend / Frontend
/ Tài liệu / Test / Cấu hình. 3. NHỊP LÀM VIỆC: commit theo tuần của từng người. Trải đều hay dồn
cục sát hạn? 4. CẢNH BÁO CẦN KIỂM CHỨNG: Commit tăng/giảm dòng đột biến (nghi template hoặc xóa
hàng loạt)? Người có net ÂM lớn? Chênh lệch commit quá lớn nghi ngờ "commit hộ"? 5. 3-5 CÂU HỎI
cần đặt cho nhóm để làm rõ.
```

1.5. Prompt 4 & 5 — Chất lượng Commit & Cơ chế Tự phản biện

```
[PROMPT 4 - Commit Message]: Đánh giá chất lượng commit message theo 3 mức (Rõ ràng / Tạm được /
Mơ hồ-vô nghĩa). Thống kê tỷ lệ và gợi ý quy ước. [PROMPT 5 - Tự phản biện]: Chỉ ra 3 kết luận mà
dữ liệu Git KHÔNG ĐỦ khẳng định. Nếu chỉ nhìn số dòng code để chấm điểm, bạn nào bị đánh giá oan
nhất, vì sao?
```

PHẦN 2: KẾT QUẢ THỰC NGHIỆM TỪ AI AGENT

Dữ liệu thực nghiệm được clone trực tiếp từ 2 repository về môi trường phân tích. AI Agent đã xử lý hơn 150 commits, hàng trăm file và bảng log numstat chi tiết.

2.1. So sánh Cấu trúc thư mục & Phân tầng kiến trúc

Đặc điểm	Nhóm 7 (IT3180_2025.2_Nhom7)	Nhóm 4 (Bai_tap_lon_IT3180_nhom_4)
Mô hình kiến trúc	**Decoupled Web Application:** Tách biệt rõ ràng Frontend (Single Page App) và Backend (RESTful Web Service).	**Monolithic Spring MVC:** Mã nguồn xử lý Backend và giao diện Template (Thymeleaf) nằm chung một dự án.
Frontend Stack	React + Vite, Tailwind CSS, nằm tại thư mục `Frontend/`. Giao tiếp qua HTTP REST và JWT Bearer Token.	Thymeleaf Server-side Rendering + Bootstrap, nằm tại `src/main/resources/templates/` và `static/`.
Backend Stack	Spring Boot 3 (Java 21) tại `Backend/`. Phân lớp chuẩn: 18 Controller, 32 Service, 27 Repository, 33 Entity, 63 DTO.	Spring Boot 3 tại `src/main/java/`. Phân gói chi tiết: Controller, Service, Service/Impl, Repository, DTO, Mapper, Scheduler.
Thư mục Tài liệu & Test	Có thư mục `docs/` riêng biệt chứa đặc tả kiến trúc. Thư mục test đặt tại `Backend/src/test`.	Không có thư mục docs riêng (tích hợp trong README). Thư mục test đặt tại `src/test`.

2.2. Tài liệu hóa dự án (Documentation)

- **Nhóm 7 — Điểm sáng về Kiến trúc tài liệu:**
  - File `README.md` (10.1 KB): Trình bày sơ đồ ASCII thể hiện cổng kết nối (Frontend port 5173, Backend port 9090, MySQL), hướng dẫn cấu hình biến môi trường, lệnh chạy môi trường dev và tài khoản mẫu (Admin/Resident).
  - File `docs/SDD.md` (191 KB): Đây là tài liệu thiết kế phần mềm (Software Design Document) đồ sộ gồm 12 chương, mô tả tường tận mô hình quan hệ cơ sở dữ liệu (ERD), bảng danh mục CSDL, tích hợp cổng thanh toán VNPay Sandbox và ma trận phân quyền.
- **Nhóm 4 — Bản hướng dẫn toàn diện nhưng dồn cục:**

- File `README.md` (20.5 KB, 1146 dòng): Nhóm đầu tư công phu với 17 mục lớn, liệt kê chi tiết 11 module nghiệp vụ chung cư, danh sách bảng database dự kiến, quy tắc đặt tên, hướng dẫn chạy Maven và cả quy ước làm việc nhóm trên Git.
- *Hạn chế:* Do dồn toàn bộ nội dung phân tích vào file README duy nhất thay vì tách thành các tài liệu kỹ thuật riêng (`docs/`), người đọc mới tiếp cận rất khó tìm nhanh hướng dẫn Quickstart để chạy thử ứng dụng.

2.3. Quy trình Git, Quản lý nhánh & Pull Request

- **Nhóm 7:**
  - Nhánh remote: `master`, `BE-develop`, `FE-develop`, `hungdev`, `khanhdev`, `kiendev`.
  - Số Pull Request (PR): Có 11 PRs (refs/pull/1..11). Các thành viên làm việc trên các nhánh tính năng/cá nhân, sau đó mở PR để merge vào nhánh phát triển chung (`BE-develop`, `FE-develop`) trước khi đưa lên `master`. Đây là quy trình Git Flow tương đối lành mạnh.
- **Nhóm 4:**
  - Nhánh remote: `main`, `feature-temporary-residence`, `feature/household-management`, `feature/setup-auth-dashboard`, `final1`.
  - Quy ước đặt tên nhánh rất chuẩn Agile (`feature/...`), tuy nhiên thực tế chỉ có 3 PRs. Phần lớn các tính năng được một thành viên chính push thẳng hoặc merge mà không qua code review thực chất.

2.4. Đánh giá Chất lượng Commit Message

Mức độ	Tiêu chí & Đặc điểm	Dẫn chứng tại Nhóm 7	Dẫn chứng tại Nhóm 4
Rõ ràng	Mô tả đúng tính năng, module hoặc lỗi được sửa; đúng ngữ cảnh.	• *"Tạo bảng hóa đơn và service tính tiền điện nước"* • *"Tích hợp VNPay sandbox và webhook IPN"* (Chiếm ~65% số commit)	• *"feat: cấu trúc Backend và giao diện trang danh sách QUẢN LÝ NHÂN KHẨU..."* • *"Complete household and fee management modules"* (Chiếm ~45% số commit)
Tạm được	Thông điệp ngắn gọn, hiểu được ngữ cảnh nhưng thiếu chi tiết.	• *"update UI resident"* • *"fix api response dto"*	• *"Them layout va dashboard co ban"* • *"add sort for tables"*
Mơ hồ / Cẩu thả	Thông điệp vô nghĩa, viết tắt, mang tính cảm xúc hoặc gộp khối code khổng lồ.	• *"fix"* • *"update"* • *"chỉnh sửa tí"*	• **"thanh vien 3"** (commit duy nhất của bạn lemanhtuanh, SHA `490ecd7`) • **"DONE 99%"** & **"99.99%DONE"** • **"1.1"** (2 commit liên tiếp SHA `26f0a54`, `8904f57`) • **"THEM CHUC NANG DANG NHAP"** (viết hoa toàn bộ)

2.5. Kiểm thử & Tích hợp liên tục (Testing & CI/CD)

> **Thực trạng đáng báo động:** Cả hai nhóm BTL kỳ trước đều nhận điểm **0/2** ở tiêu chí Kiểm thử và CI/CD:

- **Về Thư mục Test:** Thư mục test của cả 2 dự án chỉ chứa duy nhất một file mặc định sinh ra từ Spring Initializr với phương thức kiểm thử rỗng `contextLoads()` (Nhóm 7: `Backend/src/test/.../BackendApplicationTests.java`, Nhóm 4: `src/test/.../ApartmentApplicationTests.java`). Hoàn toàn không có unit test nghiệp vụ nào.
- **Về CI/CD:** Hoàn toàn không có thư mục `.github/workflows` hoặc các file cấu hình pipeline tự động build/test.

2.6. Quản lý phụ thuộc & Cấu hình .gitignore

- **Nhóm 7 — Nghịch lý về sự may mắn:** Quản lý dependency tốt thông qua `pom.xml` và `package.json`. Tuy nhiên, file `.gitignore` chỉ có duy nhất dòng `.idea/`. Việc không loại trừ `node_modules/`, `target/`, `*.class`, `.env` là một sai phạm kỹ thuật nghiêm trọng; repo không bị dính file rác chỉ là do may mắn cá nhân.
- **Nhóm 4 — Hình mẫu cấu hình .gitignore:** File `.gitignore` dài 577 bytes được thiết lập bài bản theo chuẩn Spring Boot: loại trừ triệt để `target/`, metadata của IDE (IntelliJ, Eclipse, VS Code), tệp rác hệ điều hành (`.DS_Store`) và file cấu hình nhạy cảm (`.env`, `application-local.properties`).

2.7. Nhịp làm việc & Mức độ đóng góp của thành viên

Nhóm 7: 132 Commits (09/04/2026 – 03/07/2026)

Bảng số liệu sau khi xử lý danh tính trùng lặp qua `.mailmap`:

Thành viên	Số Commit	Dòng thêm (+)	Dòng xóa (-)	Dòng Net	Nhận định vai trò & Khối lượng công việc
bkhanh2006	**91** (68.9%)	32.206	14.458	+17.748	Lực lượng nòng cốt, phụ trách chính phần Backend & kết nối cơ sở dữ liệu.
quanghung (Hung)	**21** (15.9%)	**56.362**	371	**+55.991**	Khởi tạo giao diện ban đầu (dẫn đầu dòng thêm do import template FE).
nhkiencuong	**14**	2.795	853	+1.942	Phát triển một số chức năng giao diện và logic

Thành viên	Số Commit	Dòng thêm (+)	Dòng xóa (-)	Dòng Net	Nhận định vai trò & Khối lượng công việc
(Kien)	(10.6%)				quản lý cư dân.
Off-exe	**6** (4.5%)	15.646	**54.828**	**-39.182**	Thực hiện tái cấu trúc (refactor), dọn dẹp các thư viện và template thừa.

Nhóm 4: 21 Commits (27/03/2026 – 18/06/2026)

Thành viên	Số Commit	Dòng thêm (+)	Dòng xóa (-)	Dòng Net	Nhận định vai trò & Khối lượng công việc
khanh201104	**19** (90.5%)	11.968	243	+11.725	"Gánh team": viết README, dựng layout, quản lý phí, hoàn thiện 90% hệ thống.
sapiens04	**1** (4.8%)	1.077	0	+1.077	1 commit duy nhất ngày 24/05/2026: Dựng backend & giao diện quản lý nhân khẩu.
lemanhtuanh	**1** (4.8%)	1.031	0	+1.031	1 commit duy nhất ngày 04/06/2026 mang tên *"thanh vien 3"*.

PHẦN 3: BẢNG ĐỐI SÁNH ĐỘ TRƯỞNG THÀNH KỸ THUẬT (SCORECARD)

Áp dụng khung đánh giá kỹ thuật 6 tiêu chí (mỗi tiêu chí tối đa 2 điểm) theo đúng tài liệu thực hành của Bộ môn để tính điểm tổng kết:

Hạng mục đánh giá	Nhóm 7 (BlueMoon Decoupled)	Nhóm 4 (BlueMoon Monolith)	Điểm tối đa
1. Cấu trúc & Phân tầng	**2.0 / 2.0** (Tách FE/BE hoàn chỉnh, Spring Boot phân lớp bài bản, DTO/Entity tách riêng).	**1.5 / 2.0** (Monolith MVC tổ chức package sạch sẽ, nhưng FE/BE phụ thuộc chặt, thiếu tính mô-đun).	2.0
2. Tài liệu hóa	**2.0 / 2.0** (README xuất sắc + File đặc tả kiến trúc SDD 191 KB gồm 12 chương).	**1.5 / 2.0** (README rất dài và chi tiết nhưng thiếu file đặc tả thiết kế riêng biệt).	2.0
3. Quy trình Git	**1.5 / 2.0** (Có nhánh theo vai trò, 11 PRs, nhưng commit message còn một số từ mơ hồ).	**1.0 / 2.0** (Có quy ước branch tốt, nhưng ít PR, commit dồn cục, message cẩu thả).	2.0
4. Kiểm thử (Testing)	**0.0 / 2.0** (Chỉ có 1 file test mặc định rỗng `contextLoads()`).	**0.0 / 2.0** (Chỉ có 1 file test mặc định rỗng `contextLoads()`).	2.0
5. CI/CD	**0.0 / 2.0** (Hoàn toàn không có workflow tự động).	**0.0 / 2.0** (Hoàn toàn không có workflow tự động).	2.0
6. Quản lý phụ	**1.0 / 2.0** (Dependency tốt nhưng	**2.0 / 2.0** (Dependency chuẩn và file	2.0

Hạng mục đánh giá	Nhóm 7 (BlueMoon Decoupled)	Nhóm 4 (BlueMoon Monolith)	Điểm tối đa
thuộc & .gitignore	`.gitignore` chỉ có 1 dòng, rủi ro cực cao).	`.gitignore` toàn diện, bảo vệ tốt repo).	
TỔNG ĐIỂM (Thang 12 quy về 10)	6.5 / 12 → **5.4 / 10**	6.0 / 12 → **5.0 / 10**	10.0

PHẦN 4: PHẢN BIỆN CHUYÊN SÂU & 4 CẠM BẪY PHÂN TÍCH GIT

Thực hiện theo chỉ dẫn tại *Phần C — Phản biện kết quả của AI* trong tài liệu hướng dẫn, người đánh giá phải có tư duy phản biện đối chiếu thay vì tin tuyệt đối vào các con số thống kê tự động.

4.1. Bóc tách 4 Cạm bẫy thực tế khi phân tích Repository

> **Cạm bẫy 1 — Số dòng code (LOC) là thước đo tồi:**
>
> Tại Nhóm 7, bạn *quanghung* có số dòng thêm lên tới +56.362 dòng nhưng thực chất commit ban đầu chỉ là bê nguyên template admin mã nguồn mở vào dự án. Trong khi đó, bạn *Off-exe* có số dòng net là **-39.182 dòng**. Nếu một giảng viên chấm điểm chỉ nhìn vào số dòng code, bạn *Off-exe* sẽ bị coi là phá hoại hoặc bị trừng phạt điểm số. Thực tế kiểm chứng cho thấy bạn ấy đã thực hiện công việc tái cấu trúc (refactoring) cực kỳ giá trị: **xóa bỏ hoàn toàn template rác khổng lồ để thay bằng cấu trúc code sạch tự viết**.

> **Cạm bẫy 2 — Lẫn lộn danh tính tác giả (Identity Aliasing):**
>
> Lệnh `git shortlog` ở Nhóm 7 ban đầu đếm thành 5 tác giả khác nhau dù nhóm chỉ có 4 sinh viên. Nguyên nhân là bạn *quanghung* sử dụng 2 email: một email cá nhân Gmail và một email dạng ẩn danh `noreply.github.com`. Nếu không dùng công cụ phát hiện và gộp danh tính qua file `.mailmap`, AI sẽ phân tích sai lệch toàn bộ khối lượng đóng góp của nhóm.

> **Cạm bẫy 3 — File .gitignore mỏng manh: May mắn không phải là quy trình:**
>
> Nhóm 7 không có file rác build trong cây Git dù `.gitignore` chỉ có 1 dòng `.idea/`. Đây là sự "may mắn" ngẫu nhiên chứ không thể hiện sự trưởng thành về quy trình kỹ thuật phần mềm. Ngược lại, cấu hình của Nhóm 4 mới là thực hành kỹ thuật phần mềm bền vững.

> **Cạm bẫy 4 — Hiện tượng "One-man Army" và nghi vấn "Commit hộ":**
>
> Tại Nhóm 4, bạn *khanh201104* chiếm 90.5% commit, còn 2 bạn còn lại mỗi người chỉ có đúng 1 commit trong cả kỳ (trong đó có commit *"thanh vien 3"*). Git log không thể kết luận được hai bạn còn lại có lười hay không. Có hai kịch bản cần làm rõ:
>
> > •Kịch bản A: Nhóm thực sự bỏ mặc cho 1 bạn tự làm toàn bộ.
> >
> > •Kịch bản B: Các bạn khác làm việc offline (viết tài liệu, thiết kế CSDL, test ứng dụng) và gửi mã nguồn qua tin nhắn để bạn Khánh gom lại push hộ lên GitHub.

4.2. Những gì Git KHÔNG BAO GIỜ nhìn thấy (Git Blind Spots)

- Git **không** ghi nhận công sức khảo sát nghiệp vụ, phân tích yêu cầu phần mềm và viết tài liệu Word/PDF.
- Git **không** phản ánh quá trình thiết kế giao diện Figma, chuẩn bị slide và thuyết trình đồ án.
- Git **không** thấy được các phiên lập trình cặp (Pair-programming) hoặc công sức ngồi cùng bạn để tìm và sửa lỗi logic (debugging).
- Git **chỉ** ghi nhận ai là người thực hiện lệnh commit, chứ không thể xác minh ai là người thực sự tư duy thuật toán đằng sau những dòng code đó.

4.3. Bộ câu hỏi phỏng vấn chuẩn bị cho buổi Bảo vệ Đồ án

1. **Hỏi bạn Off-exe (Nhóm 7):** *"Em có thể giải thích lý do thực hiện commit xóa hơn 50.000 dòng code trong dự án? Em đã gặp những khó khăn gì khi tái cấu trúc lại phần Frontend để tích hợp với Spring Boot?"*
2. **Hỏi bạn bkhanh2006 (Nhóm 7):** *"Em đã thiết kế cơ chế bảo mật và phân quyền giữa Admin và Resident qua JWT như thế nào? Tại sao nhóm đầu tư viết tài liệu SDD rất công phu (191 KB) nhưng lại không viết Unit test cho các Service tài chính?"*
3. **Hỏi bạn lemanhtuanh và sapiens04 (Nhóm 4):** *"Trong commit ngày 04/06 mang tên 'thanh vien 3', bạn đã chỉnh sửa những file nào? Hãy trình bày chi tiết luồng xử lý dữ liệu của chức năng Quản lý tạm trú mà*

bạn phụ trách."

4. **Hỏi bạn khanh201104 (Nhóm 4):** *"Tại sao vào ngày 09/06 nhóm lại có những commit mang tên 'DONE 99%' và '99.99%DONE' với hàng nghìn dòng code thay đổi cùng lúc thay vì tách thành các commit tính năng nhỏ lẻ? Việc phân chia công việc trong nhóm diễn ra như thế nào?"*

PHẦN 5: BÀI HỌC KINH NGHIỆM & QUY CHUẨN ÁP DỤNG CHO BTL KỲ 2026.1

Từ việc phân tích thực nghiệm và phản biện các lỗ hổng của các nhóm kỳ trước, nhóm chúng tôi thiết lập **5 quy chuẩn bắt buộc** cho bài tập lớn kỳ này:

> **1. Cấu hình .gitignore hoàn chỉnh ngay từ ngày đầu tiên:**
>
> Sử dụng template chuẩn theo ngôn ngữ (Java/Spring Boot và Node.js/React), tuyệt đối loại trừ `target/`, `build/`, `node_modules/`, logs và các file chứa thông tin nhạy cảm (`.env`, mật khẩu CSDL).

> **2. Định danh Git đồng nhất & Chuẩn hóa Commit Message:**
>
> Tất cả thành viên bắt buộc thiết lập đúng `user.name` và `user.email` trùng khớp với tài khoản GitHub. Áp dụng nghiêm ngặt chuẩn **Conventional Commits** (`feat:`, `fix:`, `docs:`, `refactor:`). Nghiêm cấm các commit vô nghĩa như "fix", "update", "1.1", "done".

> **3. Duy trì nhịp độ làm việc đều đặn & Atomic Commits:**
>
> Mỗi thành viên phải làm việc và commit mã nguồn hàng tuần theo tiến độ Sprint. Tuyệt đối không để dồn code vào 2-3 ngày trước deadline. Mỗi commit chỉ giải quyết một vấn đề cụ thể, không gộp hàng nghìn dòng code vào một lần đẩy.

> **4. Cam kết viết Kiểm thử (Unit Test) tối thiểu 20%:**
>
> Không để file test rỗng. Bắt buộc viết các ca kiểm thử đơn vị cho tầng Service và các nghiệp vụ cốt lõi (tính toán tiền phí, kiểm tra điều kiện dữ liệu).

5. Thiết lập GitHub Actions tự động kiểm thử (CI):

Xây dựng file workflow `.github/workflows/ci.yml` để tự động build và chạy test mỗi khi có Pull Request được mở, ngăn ngừa việc merge code lỗi vào nhánh chính.

Người lập báo cáo

(Ký và ghi rõ họ tên)